Source	# of Idioms	Idiom Length (words)
MAGPIE	590	4.5 ± 0.9
LIDIOMS	149	5.1 ± 1.2
EF	97	5.6 ± 1.9
EPIE	76	4.4 ± 0.7
Total (unique)	814	4.7 ± 1.8

Table 1: Statistics per data source in IDIOMEM.

a dataset of memorization-probing sequences, we will carefully curate a set of English idioms and filter out ones that do not fulfill our criteria.

3.1 The IDIOMEM Dataset

We begin with existing datasets of English idioms: MAGPIE (Haagsma et al., 2020),[5] EPIE (Saxena and Paul, 2020), and the English subset of LIDIOMS (Moussallem et al., 2018). We enrich this collection with idioms scraped from the website "Education First" (EF).[6] We then split each idiom into a prompt containing all but the last word and a target that is the last word. Next, we filter out idioms that do not comply with our criteria (§2) or whose target can be predicted from their prompt based on spurious correlations rather than memorization. To this end, we use three simple rules:

- **Short idioms**. We observe that prompts of idioms with just a few words often have multiple plausible continuations, that are not necessarily the idiom's target, violating our first criterion. For example, the prompt *"break a ____"* has many possible continuations (e.g. *"window"*, *"promise"*, and *"heart"*) in addition to its idiomatic continuation *"leg"*. To exclude such cases, we filter out idioms with < 4 words.

- **Idioms whose target is commonly predicted from the prompt's subsequence.** We filter such cases to ensure the prompt fulfills our second criterion (prompt irreducibility).

  To detect these cases, we use an ensemble of pretrained LMs: GPT2$_{\text{M}}$, ROBERTA-BASE (Liu et al., 2019), T5-BASE (Kale and Rastogi, 2020) and ELECTRA-BASE-GENERATOR (Clark et al., 2020), and check for each model if there is an n-gram ($1 \leq n \leq 4$) in the prompt from which the model predicts the target. We filter out idioms for which a majority (≥ 3) of models predicted the target (for some n-gram).

[5] We take idioms with annotation confidence of $> 75\%$ and exclude frequently occurring literal interpretations.

[6] https://www.ef.com/wwen/english-resources/english-idioms/

Prompt	Target	Pred.	Sim.	IDIOMEM
"make a mountain out of a"	molehill			✓
"think outside the"	box			✓
"there's no such thing as a free"	lunch			✓
"go back to the drawing"	board	✓		
"boys will be"	boys		✓	
"take it or leave"	it	✓	✓	

Table 2: Example English idioms included and excluded from IDIOMEM by our filters of predictable target (Pred.) and prompt-target similarity (Sim.).

- **Idioms whose targets are semantically similar to tokens in the prompt.** To further ensure prompt irreducibility, we embed the prompt's tokens and the target token using GloVe word embeddings (Pennington et al., 2014). We measure the cosine distance between the target token to each token in the prompt separately and take the maximum of all the tokens. We filter out idioms where this number is higher than 0.75 (this number was tuned manually using a small validation set of idioms).

Overall, 55.7% of the idioms were filtered out, including 48.5% by length, 6.1% by the predictable-target test, and an additional 1.6% by the prompt-target similarity, resulting in an 814 idioms dataset, named IDIOMEM. Further statistics are provided in Tab. 1, and example idioms in Tab. 2.

4 Probing Methodology

Background and Notation Assuming a transformer LM with L layers, a hidden dimension d and an input/output-embedding matrix $E \in \mathbb{R}^{|V| \times d}$ over a vocabulary V. Denote by $\mathbf{s} = \langle s_1, ..., s_t \rangle$ the input sequence to the LM, and let $\mathbf{h}_i^\ell$ be the output for token i at layer ℓ, for all $\ell \in 1, ..., L$ and $i \in 1, ..., t$. The model's prediction for a token s_i is obtained by projecting its last hidden representation $\mathbf{h}_i^L$ to the embedding matrix, i.e. $softmax(E\mathbf{h}_i^L)$.

Following (Geva et al., 2021, 2022), we interpret the prediction for a token s_i by viewing its corresponding sequence of hidden representations $\mathbf{h}_i^1, ..., \mathbf{h}_i^L$ as an evolving distribution over the vocabulary. Concretely, we read the "hidden" distribution at layer ℓ by applying the same projection to the hidden representation at that layer: $\mathbf{p}_i^\ell = softmax(E\mathbf{h}_i^\ell)$. Using this interpretation, we track the probability and rank of the predicted token in the output distribution across layers. A token's rank is its position in the output distribution

	$GPT2_M$	$GPT2_L$	$BERT_B$	$BERT_L$
Memorized idioms (`mem-idiom`)	364 44.7%	392 48.2%	230 28.3%	305 37.5%
Non-memorized idioms (`non-mem-idiom`)	450 55.3%	422 51.8%	584 71.7%	509 62.5%

Table 3: Number of memorized idioms vs. non-memorized idioms from the IDIOMEM dataset for each model. An instance is considered a memorized example if the model correctly predicts the target.

when sorted by probability from highest to lowest (e.g. the rank of the final predicted token is zero).

Probing Procedure Our key method to understand how transformer LMs retrieve information from memory is *comparing features of memory recall* to inference that does not necessarily include memory recall. Given a set of sequences that fulfill the criteria in §2, we split them into a "memorized" set whose targets' first token is predicted correctly by the model being analyzed given (and are therefore memorized), and a "non-memorized" set whose target is predicted incorrectly. We additionally include a second set of "non-memorized" instances: natural-language sequences randomly sampled from a large corpus (we assume most naturally-occurring sequences are not memorized).

To probe a LM, we run it on the 3 sets, and for each set and each layer, we (a) extract the rank and probability of the final predicted token in the hidden distribution for each prompt, and (b) compute the mean rank and probability over all prompts.

5 Probing Memorization Using Idioms

5.1 Experimental Setup

Datasets For each LM under analysis (see below), we split IDIOMEM into two disjoint subsets of memorized and non-memorized idioms, denoted as `mem-idiom` and `non-mem-idiom`, respectively, according to whether or not the LM succeeds in completing them. We produce an additional set of non-memorized instances, `wiki`, by sampling prompts from the WIKITEXT-103 dataset (Merity et al., 2017) of the same length distribution as in IDIOMEM (see Tab. 1).

Models We use multiple transformer LMs that are different in size, architecture, and optimization objective. We use GPT2 (medium and large) (Radford et al., 2019), an autoregressive transformer decoder, and BERT (base and large) (Devlin et al., 2019), a transformer encoder trained with a masked language modeling (MLM) objective. To evaluate BERT on a specific idiom, we feed the idiom's prompt concatenated with the special mask token and a period (e.g. *"think outside the* `[MASK]`."). Further details on each model are presented in Tab. 6. The number of memorized and non-memorized idioms from IDIOMEM for each model are provided in Tab. 3.

5.2 Memorized Predictions are a Two-Step Process

Fig. 2 shows the probability and rank of the output token across layers, for memorized and non-memorized idioms and short sequences from Wikipedia, by $GPT2_M$, $GPT2_L$, $BERT_B$, and $BERT_L$. Naturally, the prediction's rank decreases across layers as the prediction probability increases. However, for memorized predictions these trends occur as two distinct and sharp inference phases. In lower layers, the prediction's rank decreases from a high rank to near zero, while its probability is also close to zero. For example, in $GPT2_L$ the rank decreases until layer 20 while the probability remains below 0.1. We refer to this phase as *candidate promotion*, as the predicted token is being promoted to be a top candidate in the output distribution.

Compared to non-memorized predictions, the initial rank of memorized predictions is generally higher, especially in GPT2 (6000 vs. 3000 in $GPT2_M$, and 3000 vs. 1500 in $GPT2_L$). A potential explanation would be a generally lower frequency of the predicted token for memorized idioms. However, we find there is only low negative correlation between the initial rank of the predicted token and its frequency (see Tab. 4). We therefore offer a different explanation: non-memorized predictions are often promoted in early layers that detect local "shallow" patterns, such as common bigrams (Geva et al., 2021), while predictions for memorized idioms are not local as they requires processing of the entire input.

In the middle layers, once the predicted token reaches the top of the hidden distribution, its probability increases until the last layer. We refer to this phase as *confidence boosting*, as the distribution shifts towards the predicted token. For memorized idioms, this increase is abrupt and dramatic, with a final probability of > 0.6 across all models. In comparison, predictions on short sequences from

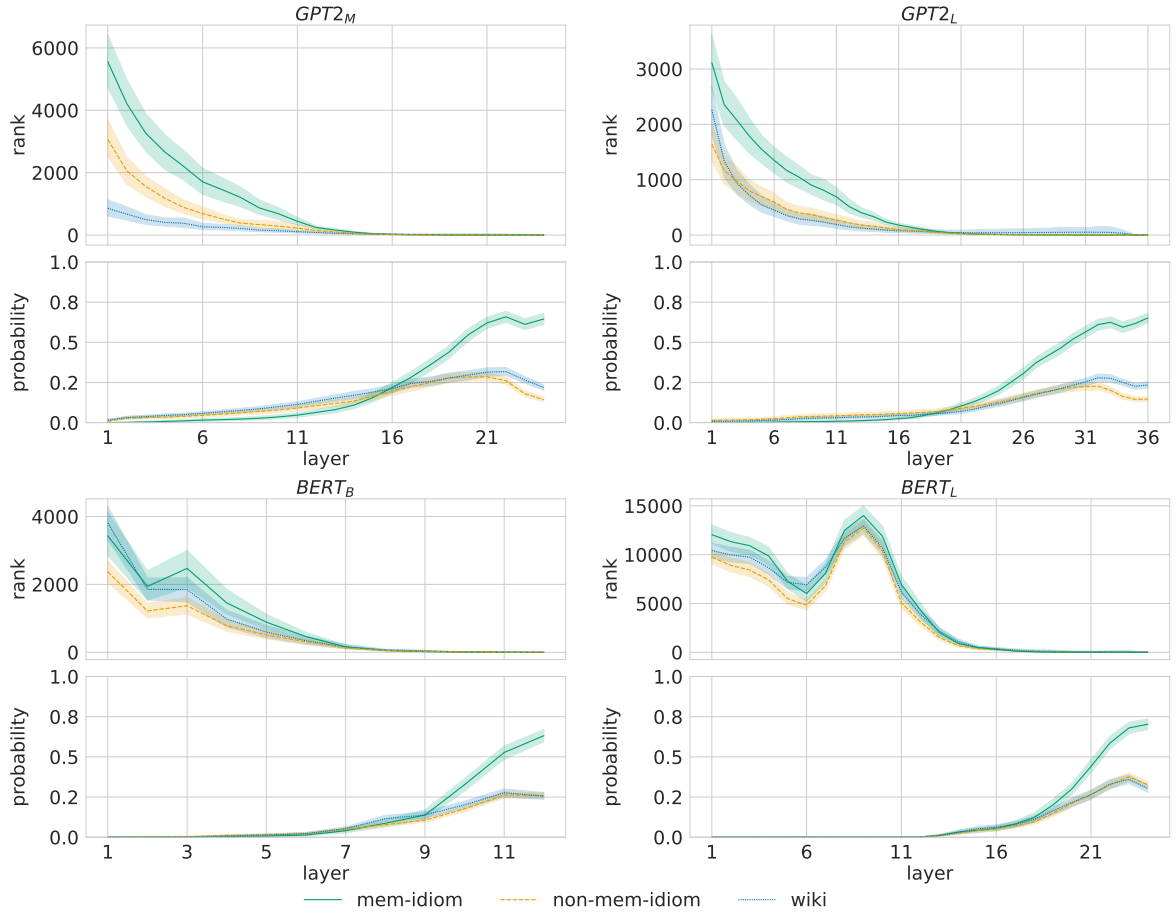

Figure 2: The predicted token's probability and rank across layers, for memorized idioms (`mem-idiom`), non-memorized idioms (`non-mem-idiom`) and short sequences from Wikipedia (`wiki`). Memory recall exhibits two characteristic phases of candidate promotion and confidence boosting.

Wikipedia and non-memorized idioms have a substantially lower probability of ~ 0.2. This can be explained by the fact that memorized idioms have a single correct target, rather than many possible continuations, as in the instances from Wikipedia. In addition, low-probability predictions for non-memorized idioms are expected as the model did not memorize the idioms and does not know their continuation. In §B, we provide more fine-grained analysis of these trends via a log-scaled view of the prediction's rank and a visualization of the ranks and probabilities for separate clusters of the memorized predictions.

We further verify that our extracted hidden-distribution features distinguish between memorized and non-memorized predictions by training linear classifiers over combinations of these features (details in §A). We observe that, indeed, our features enable separation between memorized and non-memorized predictions at high accuracy (77%-85% across models). Moreover, classifiers that use hidden distribution features are more accurate than those relying only on the model's output. Overall, these findings provide a profile of memorized predictions, suggesting that the memorized information is retrieved in early layers at inference.

	correlation	p-value
GPT2_M	-0.22	$2.9e^{-21}$
GPT2_L	-0.18	$5.4e^{-14}$
BERT_B	-0.19	$1.8e^{-15}$
BERT_L	0.15	$3.5e^{-10}$

Table 4: Pearson correlation between the predicted token's rank at the first layer and its general frequency in Wikipedia.

5.3 Testing the Roles of Different Layers Through Intervention

Our analysis in the previous section interprets hidden representations as distributions over the output vocabulary. We now conduct intervention experiments to verify that this interpretation is meaning-